MLair: A Governance-Centric ML Control Plane for Multi-Tenant AI Workloads

Phu Nguyen-Tai · Nhat Tran-Minh · Tuan Le-Anh · Bao Nguyen-Van · Quan Le-Trung

Faculty of Computer Networks and Communications
University of Information Technology — VNU-HCM

Presentation Overview

1. Motivation & Research Gap
2. MLair Design
3. Evaluation
4. Demo
5. Conclusions & Future Work

1. Motivation & Research Gap

Production ML is more than training a model

In production, operators need to answer:

- Which dataset version trained this model?
- Why was a training run accepted or blocked?
- Which model version was produced by a specific run?
- Who or what policy allowed the model to be promoted?
- How much compute did each task or tenant consume?
- How trustworthy is that resource information?

The problem is that these answers often live in different systems with different identifiers.

→ **Operational state becomes fragmented across the ML lifecycle.**

1. Motivation & Research Gap

Powerful tools — fragmented lifecycle

A typical production stack combines:

- Workflow Orchestration — Airflow / Kubernetes workflows
- Experiment Tracking — MLflow
- Artifact Storage — S3 / Object Storage
- Compute Infrastructure — Kubernetes / Cloud
- Resource & Billing Systems

But production MLOps still lacks a shared layer that jointly manages:

- DAG Run ID
- Experiment Run ID
- Dataset URI
- Model Version
- Cloud Resource Tags

→ **Different systems → different identities → manual correlation**

1. Motivation & Research Gap

The missing layer is not another scheduler

Existing platforms already provide:

- Airflow → workflow scheduling
- MLflow → experiment tracking
- Object stores → artifact persistence
- Kubernetes → compute execution

But production MLOps still lacks a shared layer that jointly manages:

- **Execution:** How and when work runs
- **Metadata:** What datasets, models, tasks, and artifacts belong together
- **Governance:** Whether work is allowed to run
- **Resource Accountability:** What resource were consumed and how trustworthy the evidence is

→ **MLair addresses this structural gap rather than claiming scheduler-throughput superiority over Airflow or experiment-browsing superiority over MLflow.**

1. Motivation & Research Gap

1 - Four-plane ML control architecture

Execution, Metadata, Governance, and Observability with resource guardrails

2 - Lifecycle-first semantics

Immutable dataset versions, readiness, eligibility, gated runs, and model governance

3 - Governance before scheduling

Invalid or ineligible workloads are rejected **before consuming execution resources**

4 - Trust-aware resource attribution

Resource Usage Contract v1 with: **TRUSTED, ADVISORY, UNTRUSTED**

5 - Controlled evaluation

Six official workload arms comparing MLair with an Airflow + MLflow baseline under frozen inputs

2. MLair Designs

MLair changes the primary abstraction

Traditional workflow-centric view:

DAG → Tasks → Execution

MLair lifecycle-centric view:

Dataset Version → Readiness → Training Eligibility → Gated Run → Model Governance

Pipeline snapshots still describe:

- DAG structure
- Dependencies
- Retries
- Replay
- Plugins

But pipelines describe **how work executes**.

The lifecycle determines **whether the work should execute and what it means**.

2. MLair Design

One control plane — four coordinated responsibilities

Excution Plane

- API admission
- Scheduler
- Executor
- Redis-backend task coordination
- Internal / external execution

Metadata Plane

- Runs and tasks
- Dataset versions
- Model versions
- Pipeline snapshots
- Lineage

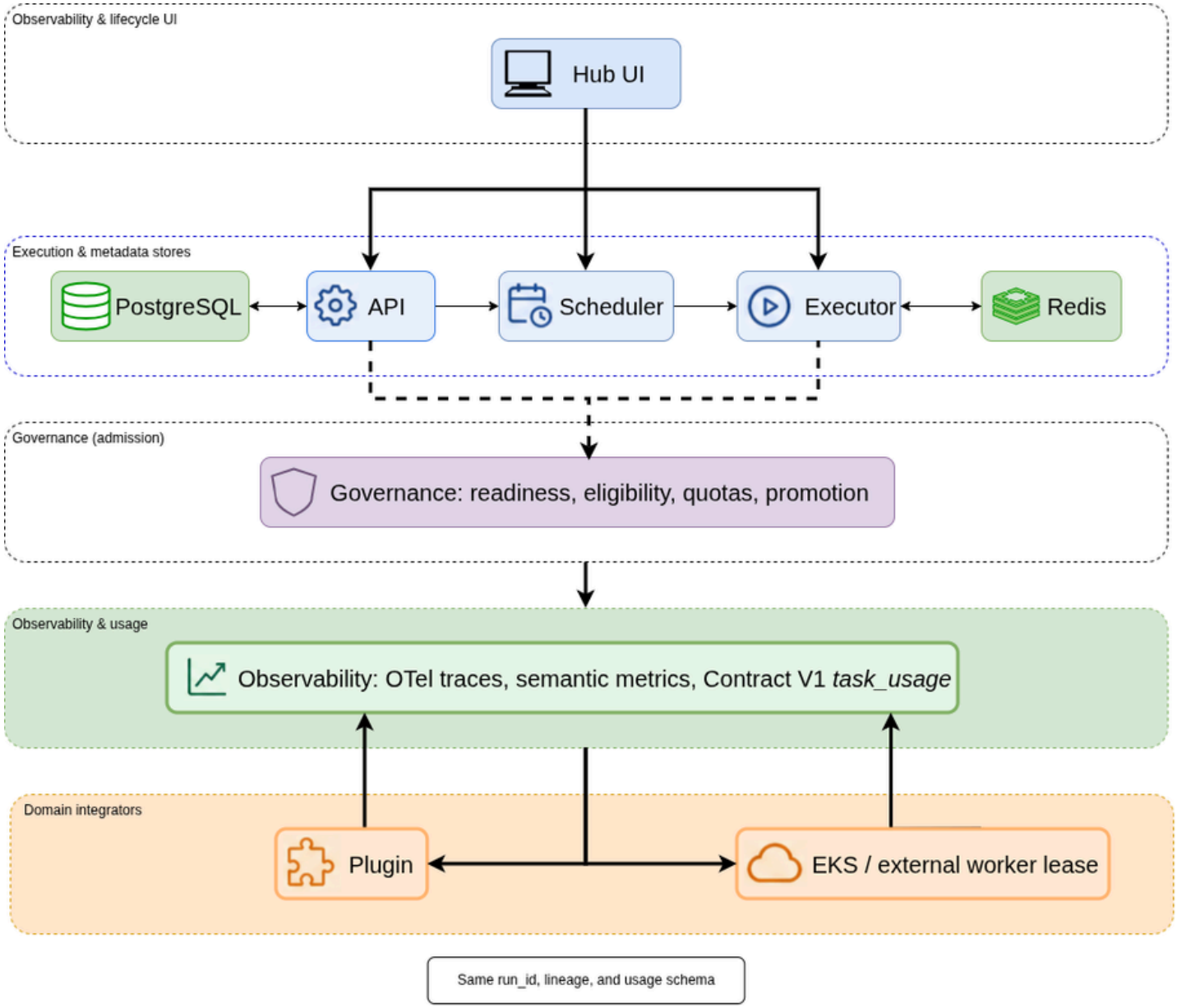
Governance Plane

- Dataset readiness
- Training eligibility
- Input contracts
- Tenant quotas
- Model promotion policy

Observability & Guardrails Plane

- Distributed traces
- Semantic metrics
- Task timelines
- Resource Usage Contract v1
- Telemetry trust classification

2. MLair Design



2. MLair Design

Control Plane

- FastAPI/v1
- Python Scheduler & Executor
- Redis
- PostgreSQL
- Next.js & React
- S3 / MinIO

Observability

- OpenTelemetry
- Prometheus
- Grafana
- WebSocket
- cgroup / procfs / psutil / NVML

Deployment

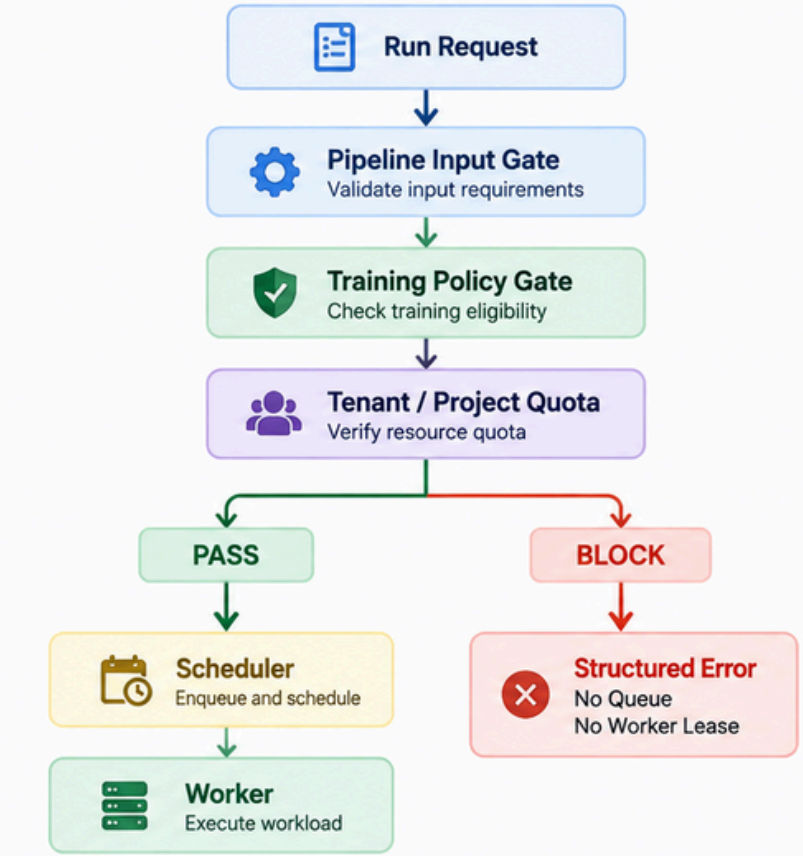
- Docker Compose
- Kubernetes + Helm
- EC2 control plane _ EKS workers

Domain Integration

- Python plugins
- HTTP Adapters
- Leased External Workers

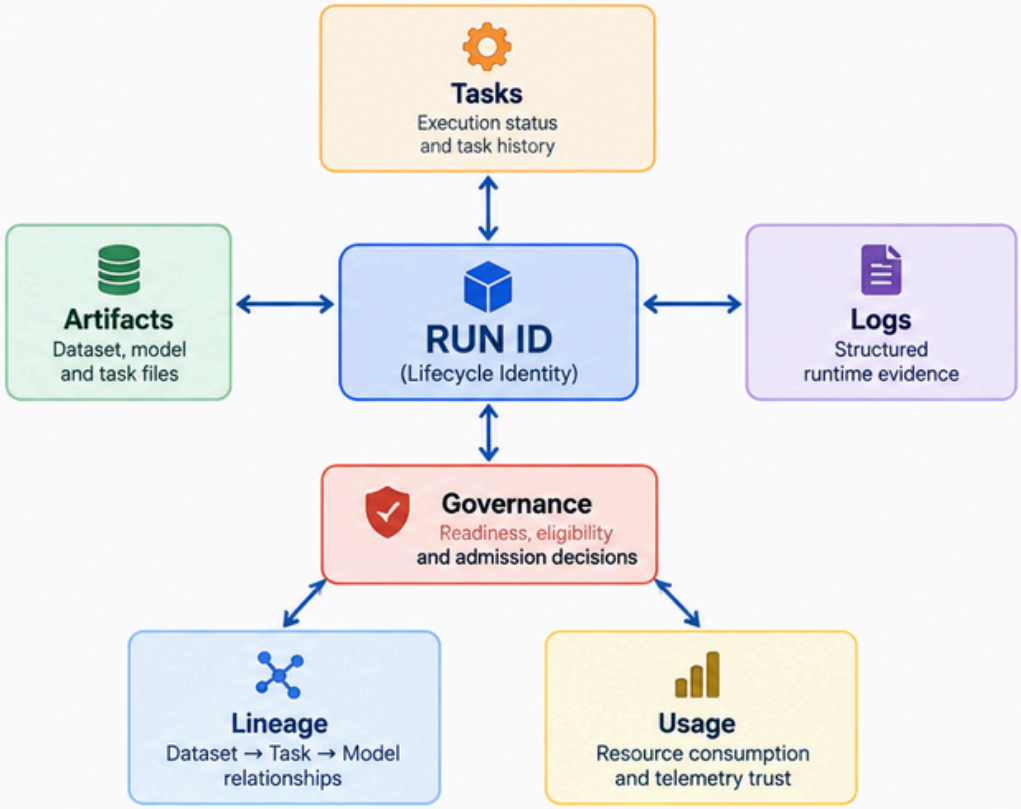
→ **MLair owns the ML control plane; domain containers own the training logic and hardware**

2. MLair Design



Governance happens before scheduling

Invalid or ineligible runs are blocked before consuming compute resources.



One Run ID for the complete lifecycle

Execution, artifacts, logs, governance, lineage, and usage are connected under a single identifier.

2. MLair Design

Same control plane — different execution environments

Local Subprocess

Build-in or local workloads executed through the internal executors. Best suited for:

- Built-in processing
- Local deployment
- Controlled execution

HTTP Plugin

MLair invokes an external domain service using the task contract. MLair retains:

- Run identity
- Task state
- Metadata
- Lineage
- Governance

Leased External Worker

Lease Task → Execute Workload → Heartbeat / Logs → Complete or Fail

2. MLair Design

Resource telemetry becomes part of the lifecycle record:

Each task can report:

- Duration
- CPU
- Memory
- GPU
- Storage
- Sample

All task-level measurements are aggregated under the same `**run_id**`.

But reporting alone is not enough

A worker can report:

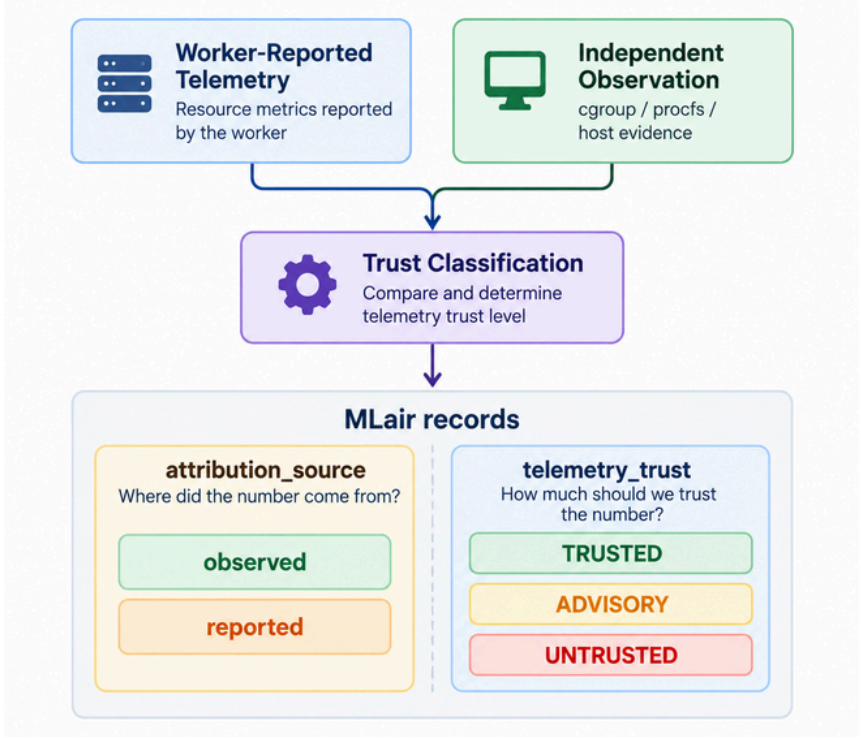
- I used 4 GB of memory

The control plane must also answer:

- How much should we trust that number?

2. MLair Design

MLair separates telemetry value from telemetry confidence



TRUSTED: Sufficient independent evidence exists and measurements are consistent.

ADVISORY: Primarily worker-reported telemetry without sufficient independent verification.

UNTRUSTED: Telemetry is missing, sparse, or inconsistent.

→ **Design Principle: Missing evidence never becomes trusted evidence.**

3. Evaluation

Two workloads families:

Bundle Y–YOLO

Arm	Stack	Execution
Y-M Internal	MLair	Internal
Y-M External	MLair	External
Y-B	Airflow + MLflow	Baseline

Bundle V–Vet-AI

Arm	Stack	Execution
V-M Internal	MLair	Internal
V-M External	MLair	External
V-B	Airflow + MLflow	Baseline

Experimental controls

- Frozen workload inputs
- Frozen hyperparameters
- Input identity checks
- Same workstation class
- Fairness gate for each arm

3. Evaluation

Evaluation metrics:

Metric	Purpose
E4: Attribution Coverage	Availability of resource usage records
E6: Execution Duration	Operational timing comparability
E7: Telemetry Trust	Reliability of resource evidence
Metadata Cohesion	Unified lifecycle view under one Run ID

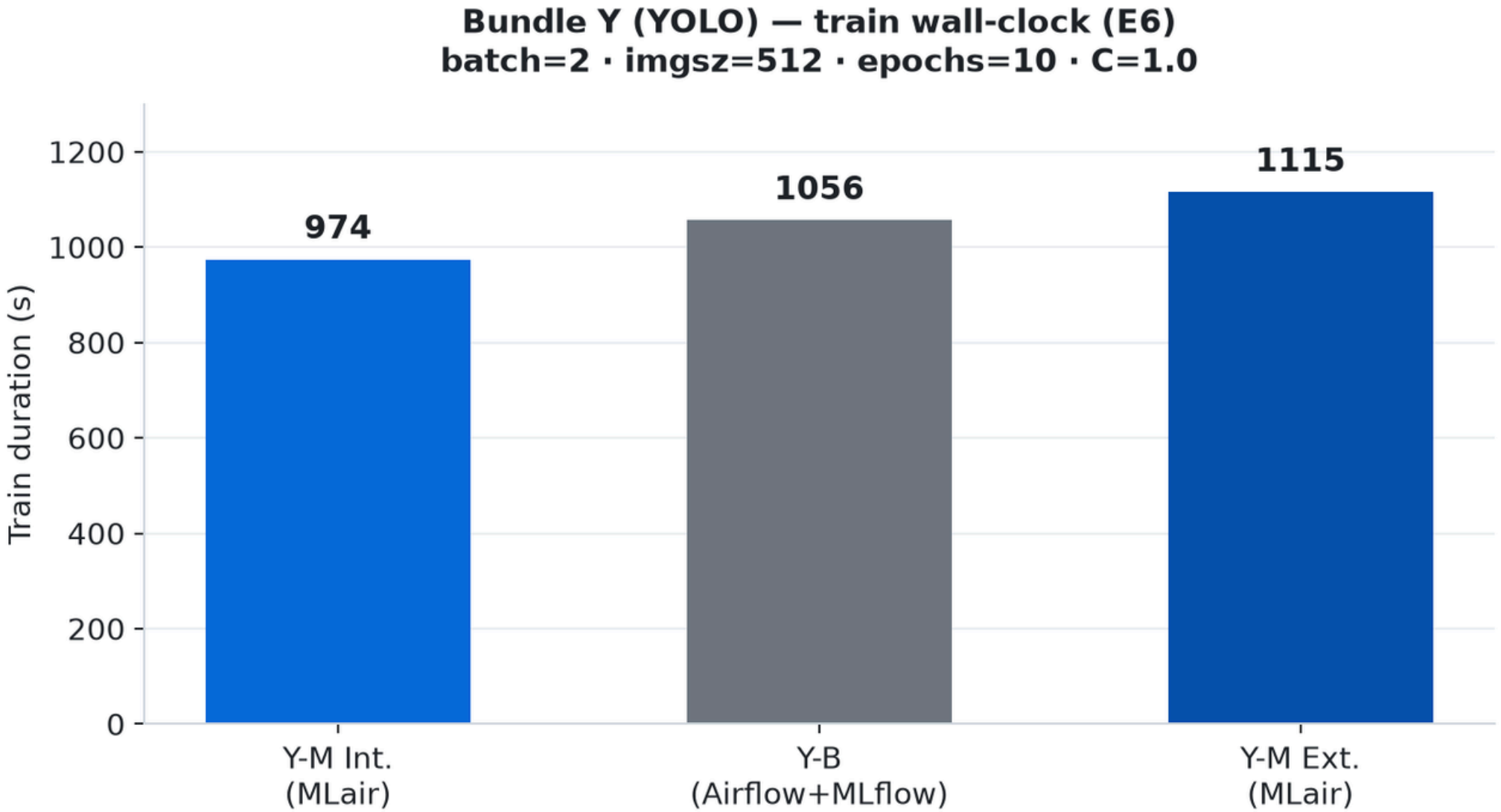
Scope:

- Lifecycle governance
- Lineage & accountability
- Resource attribution

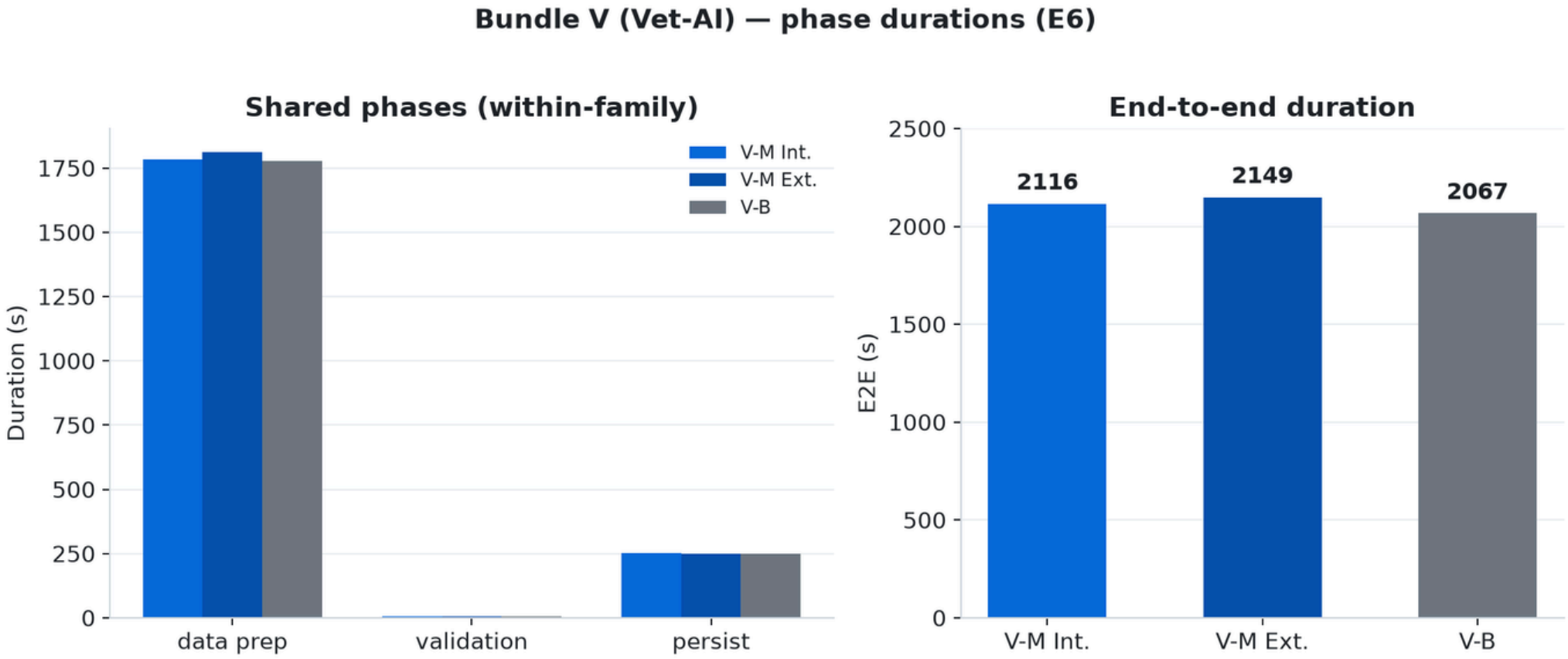
Not evaluated:

- Scheduler throughput
- Production-scale concurrency

3. Evaluation



3. Evaluation



Observation

Data preparation, validation, and persistence remain approximately aligned across stacks

End-to-end

- MLair Internal: 2116 s
- Baseline: 2067 s

≈ 2.4% difference

3. Evaluation

Execution alone is not MLair's main advantage

Capacity	Baseline	MLair
Task State	✓	✓
Parameters / Metrics	✓	✓
Artifacts	△	✓
Dataset Lineage	△	✓
Model Lineage	△	✓

Capacity	Baseline	MLair
Admission / Quota	×	△
Lifecycle Gate Stask	×	✓
Resource Usage	✓	✓
Trust-Labeled Attribution	×	✓
Single Run ID	×	✓

Baseline workflow

Operator correlates:

Airflow Task Instance + MLflow Run + Artifact Paths + additional resource evidence

3. Evaluation

Demonstrated

- ✓ Lifecycle-first orchestration
- ✓ Governance gates before scheduling
- ✓ Internal and external execution
- ✓ Unified dataset/model lineage
- ✓ Resource Usage Contract v1
- ✓ Trust-aware resource attribution
- ✓ One run ID across operational evidence
- ✓ Comparable within-family workload timings

Not Demonstrated

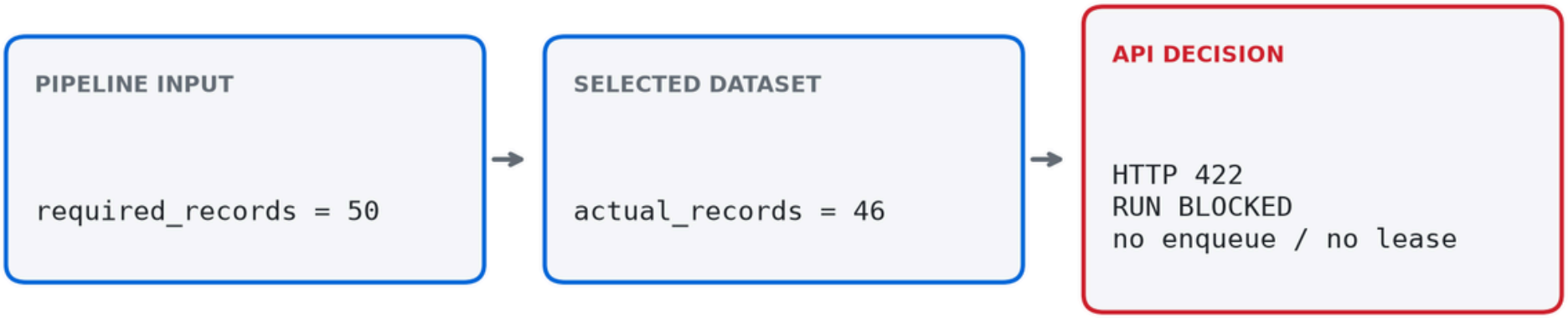
- ✗ Production-scale scheduler throughput
- ✗ Large-scale concurrency
- ✗ Isolated governance-gate overhead
- ✗ Full external-worker attestation
- ✗ Dollar-based chargeback
- ✗ Cross-workload performance superiority

4. Demo

From dataset to auditable execution

1. **Dataset Readiness**
   - Evaluate whether the dataset can be used for training.
2. **Gated Run Admission**
   - Apply policy and pipeline input requirements.
3. **Run Execution**
   - Inspect tasks, logs, status, and metrics.
4. **Lineage**
   - Trace dataset → task → model relationships.
5. **Usage & Trust**
   - Inspect task-level resource consumption and telemetry quality.

4. Demo



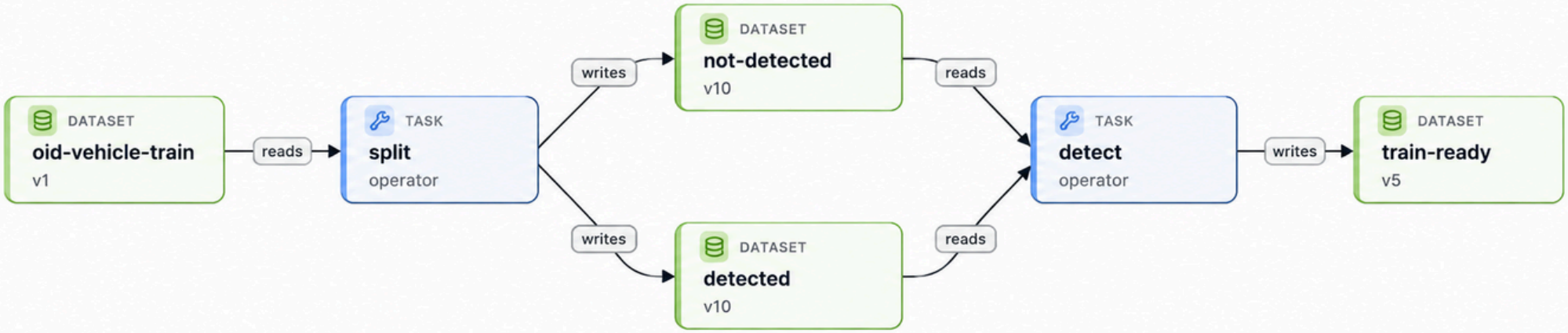
Governance before scheduling

Dataset undersized → structured block → no scheduler work

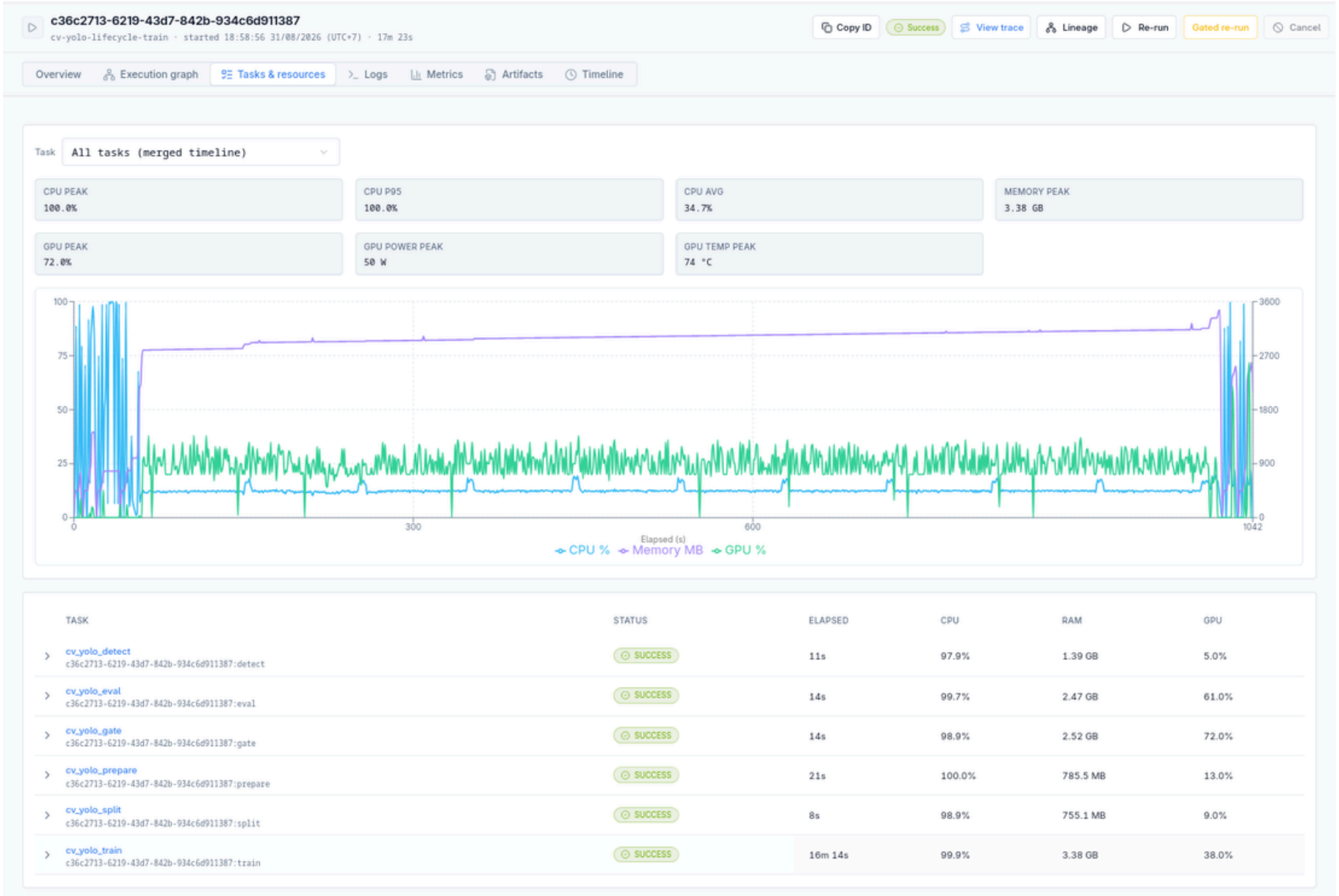
Demo A — readiness / admission gate (slide 24)

Hub: http://127.0.0.1:8080

4. Demo


DATASET
oid-vehicle-train
v1
reads
TASK
split
operator
writes
writes
DATASET
not-detected
v10
DATASET
detected
v10
reads
reads
TASK
detect
operator
writes
DATASET
train-ready
v5

4. Demo


c36c2713-6219-43d7-842b-934c6d911387
cv-yolo-lifecycle-train · started 18:58:56 31/08/2026 (UTC+7) · 17m 23s
Copy ID
Success
View trace
Lineage
Re-run
Gated re-run
Cancel
Overview
Execution graph
Tasks & resources
Logs
Metrics
Artifacts
Timeline
Task
All tasks (merged timeline)
CPU PEAK
100.0%
CPU P95
100.0%
CPU AVG
34.7%
MEMORY PEAK
3.38 GB
GPU PEAK
72.0%
GPU POWER PEAK
50 W
GPU TEMP PEAK
74 °C
100
75
50
25
0
3600
2700
1800
900
0
0
300
600
1042
Elapsed (s)
CPU %
Memory MB
GPU %
TASK
STATUS
ELAPSED
CPU
RAM
GPU
cv_yolo_detect
c36c2713-6219-43d7-842b-934c6d911387:detect
SUCCESS
11s
97.9%
1.39 GB
5.0%
cv_yolo_eval
c36c2713-6219-43d7-842b-934c6d911387:eval
SUCCESS
14s
99.7%
2.47 GB
61.0%
cv_yolo_gate
c36c2713-6219-43d7-842b-934c6d911387:gate
SUCCESS
14s
98.9%
2.52 GB
72.0%
cv_yolo_prepare
c36c2713-6219-43d7-842b-934c6d911387:prepare
SUCCESS
21s
100.0%
785.5 MB
13.0%
cv_yolo_split
c36c2713-6219-43d7-842b-934c6d911387:split
SUCCESS
8s
98.9%
755.1 MB
9.0%
cv_yolo_train
c36c2713-6219-43d7-842b-934c6d911387:train
SUCCESS
16m 14s
99.9%
3.38 GB
38.0%

5. Conclusions

1. Lifecycle First

MLair models:

- dataset → readiness → training → model

rather than treating ML workloads only as DAGs.

2. Governance Before Execution

Policies and input contracts are evaluated before work reaches compute resources.

3. One Run ID

Execution, logs, artifacts, lineage, governance, and resource usage share a common lifecycle identity.

4. Trust-Aware Telemetry

Resource values are accompanied by explicit evidence quality:

TRUSTED · ADVISORY · UNTRUSTED

5. Conclusions

Current Limitations

Scale

- No production-scale scheduler concurrency benchmark

Governance Performance

- Admission-gate overhead has not been isolated

External Execution

- Local external-worker lease evidence remains incomplete

Resource Accounting

- Telemetry trust exists, but MLair does not yet provide complete dollar **chargeback**

Evaluation

- Controlled local campaign
- No cross-family workload comparison

Future Work

Pricing & Cost Attribution

- Add pricing adapters on top of per-task resource usage.

Stronger External Verification

- Improve validation of worker-reported telemetry and lease evidence.

Concurrency Evaluation

- Measure scheduler and API behavior at higher multi-tenant scale.

Production Validation

- Expand evaluation across heterogeneous infrastructure and larger deployments.

THANK YOU FOR LISTENING

Phu Nguyen-Tai · Nhat Tran-Minh · Tuan Le-Anh · Bao Nguyen-Van · Quan Le-Trung

Faculty of Computer Networks and Communications
University of Information Technology — VNU-HCM